# ADS CMCE Practice Quiz 3

Total points 22/40

The respondent's email (**rushikesh.sambare.cmfeb26@gmail.com**) was recorded on submission of this form.

0 of 0 points

PRN *

260240320142

Name: *

Rushikesh Bhagwat Sambare

MCQ

22 of 40 points

✓ What is recurrence for worst case of QuickSort and what is the time complexity in Worst case? *1/1

- ○ Recurrence is T(n) = T(n-2) + O(n) and time complexity is O(n^2)
- ◉ Recurrence is T(n) = T(n-1) + O(n) and time complexity is O(n^2) ✓
- ○ Recurrence is T(n) = 2T(n/2) + O(n) and time complexity is O(nLogn)
- ○ Recurrence is T(n) = T(n/10) + T(9n/10) + O(n) and time complexity is O(nLogn)

✕ What is the best time complexity of bubble sort(optimised)? * 0/1

- ○ O(n^2)
- ◉ O(n log n) ✕
- ○ O(n)
- ○ O(nlog n^2)

Correct answer

- ◉ O(n)

✕ Which of the following sorting algorithms is the fastest for sorting small arrays? *0/1

- ◉ a) Quick sort ✕
- ○ b) Shell sort
- ○ c) Insertion sort
- ○ d) Heap sort

Correct answer

- ◉ c) Insertion sort

✕ What is the advantage of selection sort over other sorting techniques? * 0/1

- (●) a) It is faster than any other sorting technique ✕
- ( ) b) It is scalable
- ( ) c) It works best for inputs which are already sorted
- ( ) d) It requires no additional storage space

Correct answer

- (●) d) It requires no additional storage space

✓ Quick sort uses which of the following method to implement sorting? * 1/1

- (●) a) partitioning ✓
- ( ) b) selection
- ( ) c) exchanging
- ( ) d) merging

✕ Which sorting algorithm is the most suitable for **linked lists**? * 0/1

- ○ a) Merge Sort
- ○ b) Quick Sort
- ○ c) Heap Sort
- ◉ d) Bubble Sort ✕

Correct answer

- ◉ a) Merge Sort

✕ The main advantage of **Quick Sort** over Merge Sort is: * 0/1

- ○ a) Quick Sort is stable
- ○ b) Quick Sort works in-place (uses less memory)
- ◉ c) Quick Sort is always faster ✕
- ○ d) Quick Sort has O(n) complexity

Correct answer

- ◉ b) Quick Sort works in-place (uses less memory)

✕ In Bubble Sort, the maximum number of comparisons in the worst case for n elements is: *0/1

- (●) a) O(n) ✕
- ( ) b) O(log n)
- ( ) c) O($n^2$)
- ( ) d) O(n log n)

Correct answer

- (●) c) O($n^2$)

✓ What is the output of the following code * 1/1

```
int arr[] = {5, 3, 4, 1, 2};
for (int i = 0; i < arr.length - 1; i++) {
   for (int j = 0; j < arr.length - i - 1; j++) {
      if (arr[j] > arr[j + 1]) {
         int temp = arr[j];
         arr[j] = arr[j + 1];
         arr[j + 1] = temp;
      }
   }
}
System.out.println(arr[2]);
```

- ( ) 4
- ( ) 5
- (●) 3 ✓
- ( ) 2

✕ What is the output for the following code? * 0/1

```
int[] a = {10, 7, 8, 9, 1, 5};

Arrays.sort(a);

System.out.println(a[a.length - 2]);
```

- ○ 10
- ○ 9
- ◉ 8 ✕
- ○ 7

Correct answer

- ◉ 9

✕ What is the **worst-case time complexity** of Merge Sort? * 0/1

- ○ a) O(n log n)
- ○ b) O($n^2$)
- ○ c) O(log n)
- ◉ d) O(n) ✕

Correct answer

- ◉ a) O(n log n)

✕ The **worst-case** performance of Quick Sort occurs when: * 0/1

- (●) a) All elements are distinct ✕
- ( ) b) The pivot is always the middle element
- ( ) c) The array is already sorted (ascending or descending)
- ( ) d) The pivot is chosen randomly

Correct answer

- (●) c) The array is already sorted (ascending or descending)

✓ Bubble Sort works by repeatedly: * 1/1

- ( ) a) Dividing the array into two halves
- (●) b) Swapping adjacent elements that are in the wrong order ✓
- ( ) c) Picking the smallest element and placing it first
- ( ) d) Selecting the pivot element

✓ Heap Sort is **not stable** because: * 1/1

- ( ) a) It uses divide and conquer
- (●) b) It swaps non-adjacent elements ✓
- ( ) c) It requires extra space
- ( ) d) It uses recursion

✗ A tree with n vertices has how many edges? * 0/1

- a) n
- b) n − 1
- c) n + 1
- d) $n^2$ ✗

Correct answer

- b) n − 1

✓ In a tree, nodes with **no children** are called: * 1/1

- a) Root nodes
- b) Internal nodes
- c) Leaf nodes ✓
- d) Branch nodes

✓ A **binary tree** is a tree where: * 1/1

- a) Each node can have at most two children ✓
- b) All nodes have two children
- c) The root has two children
- d) All leaves are at the same depth

✓ A **complete binary tree** is one in which: * 1/1

- ○ a) Every level is filled completely
- ◉ b) Every level except possibly the last is full, and the last level is filled from left to right ✓
- ○ c) All leaves are at the same level
- ○ d) Each node has two children

✓ In-order traversal of a binary tree results in: * 1/1

- ◉ a) Left → Root → Right ✓
- ○ b) Root → Left → Right
- ○ c) Right → Left → Root
- ○ d) Root → Right → Left

✓ Post-order traversal of a binary tree results in: * 1/1

- ○ a) Root → Left → Right
- ○ b) Left → Root → Right
- ◉ c) Left → Right → Root ✓
- ○ d) Right → Root → Left

✕ The **time complexity** of search, insertion, and deletion in a BST (average case) is: *0/1

- ○ a) O(1)
- ○ b) O(log n)
- ◉ c) O(n) ✕
- ○ d) O(n log n)

Correct answer

- ◉ b) O(log n)

✕ The **worst-case** time complexity of BST operations occurs when: * 0/1

- ○ a) Tree is balanced
- ○ b) Tree is skewed (like a linked list)
- ◉ c) All nodes have two children ✕
- ○ d) Tree has duplicate keys

Correct answer

- ◉ b) Tree is skewed (like a linked list)

✓ A **balanced binary tree** is one where: * 1/1

- (●) a) Left and right subtrees of every node differ in height by at most one ✓
- ( ) b) All leaf nodes are at the same level
- ( ) c) Each internal node has two children
- ( ) d) The tree height equals $\log_2(n)$

✓ An **AVL Tree** maintains: * 1/1

- ( ) a) Equal number of nodes in left and right subtrees
- (●) b) Balance factor (height difference) ≤ 1 for every node ✓
- ( ) c) Level-wise order of nodes
- ( ) d) Heap property

What is the output of the following code? * 1/1

```
static void in(Node n){
    if(n==null) return;
    in(n.l); System.out.print(n.d+" "); in(n.r);
}
public static void main(String[] args){
    // Tree:    1
    //        /   \
    //       2     3
    //     /  \     \
    //    4    5     6
    Node root=new Node(1);
    root.l=new Node(2); root.r=new Node(3);
    root.l.l=new Node(4); root.l.r=new Node(5);
    root.r.r=new Node(6);
    in(root);
}
```

- a) 1 2 4 5 3 6
- b) 1 2 3 4 5 6
- c) 4 2 5 1 3 6 ✓
- d) 4 5 2 6 3 1

✓ What is the output? * 1/1

```
static void post(Node n){
   if(n==null) return;
   post(n.l); post(n.r); System.out.print(n.d+" ");
}
public static void main(String[] args){
   Node root=new Node(1);
   root.l=new Node(2); root.r=new Node(3);
   root.l.l=new Node(4); root.l.r=new Node(5);
   root.r.r=new Node(6);
   post(root);
}
```

- ○ a) 1 2 4 5 3 6
- ◉ b) 4 5 2 6 3 1 ✓
- ○ c) 2 4 5 1 3 6
- ○ d) 1 3 6 2 5 4

✗ A **stack** works on which principle? * 0/1

- ○ a) FIFO
- ◉ b) LIFO ✗
- ○ c) FILO
- ○ d) Both b and c

Correct answer

- ◉ d) Both b and c

✓ What happens when you try to **pop** from an empty stack? * 1/1

- (●) a) Underflow ✓
- ( ) b) Overflow
- ( ) c) Runtime Error
- ( ) d) Compilation Error

✓ What is the time complexity of **push()** and **pop()** in a stack (array or linked list implementation)? *1/1

- ( ) a) O(log n)
- ( ) b) O(n)
- (●) c) O(1) ✓
- ( ) d) $O(n^2)$

✓ Which of the following is **not** an application of stacks? * 1/1

- ( ) a) Function call management
- ( ) b) Expression evaluation
- (●) c) Level-order traversal ✓
- ( ) d) Undo mechanism in editors

✕ Given the expression: * 0/1

A + B * (C – D)

What is the corresponding **Postfix** form?

- a) A B C D – * +
- b) A B + C D * –
- c) A B + C D – *
- d) A B C D + * – ✕

Correct answer

- a) A B C D – * +

✓ What is the output? * 1/1

```
Stack<Integer> s = new Stack<>();
s.push(10); s.push(20); s.pop(); s.push(30);
System.out.println(s.peek());
```

- a) 10
- b) 20
- c) 30 ✓
- d) Empty Stack Exception

✕ Which algorithm typically uses a **stack**? * 0/1

- ○ a) BFS
- ○ b) DFS
- ○ c) Kruskal's algorithm
- ◉ d) Dijkstra's algorithm ✕

Correct answer

- ◉ b) DFS

✓ What happens when you try to enqueue into a full queue? * 1/1

- ○ a) Underflow
- ◉ b) Overflow ✓
- ○ c) Runtime Error
- ○ d) Memory Leak

✓ Which of the following queue types allows **insertion and deletion from both ends**? *1/1

- ○ a) Simple Queue
- ○ b) Circular Queue
- ○ c) Priority Queue
- ◉ d) Deque ✓

✗ Which of the following is **not** an application of queues? * 0/1

- ◉ a) CPU scheduling ✗
- ○ b) Disk scheduling
- ○ c) Function recursion
- ○ d) Print spooling

Correct answer

- ◉ c) Function recursion

✕ In a **priority queue**, elements are removed based on: * 0/1

- ○ a) Order of insertion
- ○ b) Value priority
- ◉ c) Both ✕
- ○ d) Random choice

Correct answer

◉ b) Value priority

✕ Which linked list is most suitable for implementing **undo and redo** operations? *0/1

- ○ a) Singly linked list
- ○ b) Doubly linked list
- ◉ c) Circular singly linked list ✕
- ○ d) Linear array

Correct answer

◉ b) Doubly linked list

✓ Which operation is **more efficient** in a linked list than an array? * 1/1

- ○ a) Searching
- ○ b) Accessing the ith element
- ◉ c) Insertion and deletion ✓
- ○ d) Sorting

✓ Prim's Algorithm is used to find: * 1/1

- ○ a) Shortest path between two vertices
- ◉ b) Minimum Spanning Tree (MST) of a weighted graph ✓
- ○ c) Strongly connected components
- ○ d) Topological ordering

This content is neither created nor endorsed by Google. - Contact form owner - Terms of Service - Privacy Policy

Does this form look suspicious? Report

Google Forms